Apresentação preliminar 05/11 — roteiro de fala, tempos e perguntas prováveis

Formato: **5 min + 2 min de perguntas**, até 5 slides, **todos falam**, sem demonstração obrigatória, mas com **evidência de que a topologia sobe**. Avaliação: clareza · coerência com o que foi construído · **honestidade sobre o que falta** · qualidade do plano.

1. Divisão e tempos (ensaiar com cronômetro: alvo 4min40s)

Slide	Quem	Tempo	Mensagem central (1 frase)
1 Arquitetura e SPOF	Integrante 1 (Infra)	60 s	"Os nós Rust são redundantes; o Nginx, o watchdog, o Prometheus e o próprio host não são."
2 O que já roda	Integrante 2 (Aplicação)	75 s	"Sobe com um comando, reparte 10/10, e um nó não pronto sai do tráfego sem reiniciar."
3 Métricas e alertas	Integrante 3 (Observabilidade)	60 s	"Definimos o que é 'estar bom' antes dos alertas, e cada limiar tem um porquê."
4 Hipóteses e experimentos	Integrante 4 (Relator)	60 s	"A primeira hipótese foi confirmada com números, e o teste revelou que monitor e balanceador discordam por até ~8 s."
5 Riscos e plano	Todos (1 frase cada)	45 s	"Achamos e corrigimos 5 problemas; o maior risco restante é medir o erro no nó e não na borda."

(Equipe menor: acumular — ex.: 1+2 e 3+4.)

2. Falas sugeridas

Slide 1 (60 s). "O cliente fala só com o Nginx, na porta 8080. O Nginx reparte em revezamento entre Rust A e Rust B. O watchdog, independente, sonda o `/health/ready` de cada nó a cada 5 s e, além disso, faz um teste sintético passando pelo Nginx — o caminho real do usuário. O Prometheus coleta métricas de todos a cada 5 s. Em vermelho estão os pontos únicos de falha: se o Nginx cair, o serviço inteiro cai mesmo com os dois nós saudáveis. O watchdog é SPOF do monitoramento — por isso o Prometheus tem um alerta sobre ele. E o Prometheus não tem ninguém vigiando: é o SPOF que declaramos. Tudo roda numa máquina só, então as falhas não são independentes."

Slide 2 (75 s). "Isto é a saída real do `docker compose ps` de 02/11: cinco serviços saudáveis, e só duas portas no host — a 8080 do Nginx e a 9090 do Prometheus presa em 127.0.0.1. Vinte requisições: dez para cada nó. Cada requisição recebe um `request_id` do Nginx, que aparece também no log do Rust — é assim que correlacionamos. Liveness e readiness são diferentes: quando desligamos a prontidão do A, o `live` continua 200, o `ready` vira 503, e o usuário continua recebendo 200 — o log do Nginx mostra `503, 200`: tentou o A, foi recusado, refez no B. Nada foi reiniciado."

Slide 3 (60 s). "Seguimos a regra 1: SLI antes do alerta. Escolhemos 5 métricas que cobrem os quatro sinais clássicos e a saúde do próprio monitor. Os três alertas obrigatórios: NoDown, que só dispara depois de três falhas de 5 s, para não alarmar por perda isolada; taxa de erro acima de 2% em 2 minutos; e monitor parado, quando a última sondagem tem mais de 30 s —

seis intervalos. Cada alerta tem severidade, dono e procedimento escritos no arquivo de regras. Sem Alertmanager: os alertas aparecem na página do Prometheus — limitação declarada."

Slide 4 (60 s). "Hipótese 1: parar o A não derruba o serviço. Previmos de 1 a 3 requisições lentas de ~1 s, que é o connect timeout. Observado: 20 de 20 com 200, três a 1,09 s, watchdog DOWN em 14 s, alerta ativo em 22 s, e depois do `start` tudo resolvido, sem reinício. Mas testar a recuperação mostrou algo que não previámos: o watchdog vê o A de volta em 2 a 3 s, e o Nginx pode levar até ~10 s para mandar tráfego para ele — porque ele re-resolve o DNS num ciclo de 10 s; medimos 4,6, 9,4 e 10,5 s. Monitor e balanceador discordam. Os próximos experimentos — nó lento, watchdog parado, erro 500 — vão justamente explorar essas discordâncias."

Slide 5 (45 s, uma frase por pessoa).

1. "Achamos cinco problemas nos nossos testes — timeout que não cobria DNS, dois alertas com falso positivo, healthcheck dependente e relógio que salta — e guardamos o antes e o depois."
2. "O maior risco técnico é medir a taxa de erro no nó: o que o usuário vê só o Nginx sabe."
3. "Plano: experimentos da Hora 6 até 08/11, relatório e README até 10/11."
4. "Ensaio cronometrado e vídeo de plano B em 11/11; entrega em 12/11."

3. Evidência de que a topologia sobe (levar pronta)

- Print/terminal com `docker compose ps` **com horário** (`date; docker compose ps`).
- `evidencias/preliminar/01_compose_ps.txt` e `04_balanceamento_20req.txt` abertos.
- Opcional, se pedirem ao vivo: `docker compose up -d && scripts/teste_sintetico.sh 6` (subida em ~20 s com imagens já construídas).

4. Perguntas prováveis e respostas curtas

Pergunta	Resposta (com a evidência)
O Nginx é SPOF? Como resolveriam?	Sim. Dois Nginx com IP virtual (keepalived/VRRP) ou um LB gerenciado. Aqui só detectamos: teste sintético + alerta crítico.
Quem vigia o vigia?	O Prometheus vigia o watchdog (`MonitorParado`, idade da última sondagem > 30 s, e `ColetaFalhando`). O Prometheus ninguém vigia — SPOF residual; em produção, *dead man's switch* externo.
Por que DOWN só após 3 falhas?	Regra 6: não decidir com uma amostra. 3 × 5 s ≈ 15 s; medido: 14,4 s. Trade-off: 15 s de detecção × menos falso alarme.
Diferença entre liveness e readiness no código?	`/health/live` só responde — nunca atrasa nem depende de nada; o Docker usa ele. `/health/ready` reflete prontidão e o atraso artificial; o watchdog usa ele. Evidência 03: live 200 com ready 503.

Pergunta	Resposta (com a evidência)
O Nginx sonda o `/health/ready`?	Não: o Nginx open source só faz verificação **passiva**. Por isso um nó não pronto responde 503 também em `/`; o Nginx refaz no outro e, após 3 falhas, o retira por 10 s.
Retry não amplifica carga?	Sim, por isso: só 2 tentativas, só em erro de conexão/timeout/502/503, nunca em 500, teto de 4 s. E só GET (idempotente). O experimento E2b vai medir isso.
Por que 1 s de connect timeout?	Na rede local conectar leva < 1 ms; 1 s é folga enorme e limita o custo de um nó parado a ~1 s (medido: 1,09 s).
Por que 2% de erro?	Abaixo, com este volume, é ruído (1 em 50); acima, o usuário percebe. Janela de 2 min suaviza picos.
Por que o tráfego demora a voltar?	`resolve` + `resolver valid=10s`: o contêiner parado some do DNS e o Nginx tira o servidor; ele só volta na próxima re-resolução. Como o ciclo não está sincronizado com o `start`, o atraso vai de 0 a ~10 s: medimos ≤ 4,6 s, 9,4 s e 10,5 s. Vamos testar `valid=5s` (E6).
Por que usar `resolve`?	Sem ele, o Nginx resolve só na inicialização: se um nó estiver fora na hora, o Nginx **não sobe** (evidência 06: `host not found in upstream`), e se o nó voltar com outro IP, nunca é encontrado.
Por que, no arquivo 04, as 20 requisições foram para o B?	Achado 8: o teste anterior deixou o A não pronto, e a sonda sintética recebeu 503 e renovou o `fail_timeout=10s`. Quando medimos, o A já estava pronto, mas ainda fora do pool. O 10/10 está no `05`, depois da recuperação. Corrigimos o script e mantivemos o arquivo original como evidência.
O que o falso positivo de erro ensinou?	Que métrica do nó mede tentativas, não o que o usuário recebeu. 503 refeito em B virou "erro" mas o usuário viu 200. Corrigimos o numerador; o certo é medir na borda — melhoria priorizada.
E os saltos de relógio?	Medimos −2,4 s/+5,1 s no WSL2, no host e no contêiner (mesmo kernel). Correlacionamos por `request_id`; latências usam relógio monotônico; limiar de 30 s tem folga.
O que falta?	Experimentos E2–E4 e E6, incidente (Hora 7), relatório F, SLI na borda, ensaio da demo. Está no slide 5 e em `docs/E_matriz_experimentos.md`.

Pergunta	Resposta (com a evidência)
O que este laboratório não prova?	Falhas independentes de rede/máquina, partição real, carga real, perda de pacotes entre hosts. Tudo divide um kernel, um Docker e um relógio.
Usaram o kit do professor?	O kit não estava no repositório; implementamos as rotas do Apêndice B em Rust (axum) e o watchdog em Python só com biblioteca padrão, mais `request_id` no log e teste sintético (desafios extras).

5. Checklist antes de apresentar

- ☐ `docker compose down && docker compose up -d --build && scripts/teste_sintetico.sh 6` no dia
- ☐ Print de `date; docker compose ps` com horário (plano B se não houver terminal)
- ☐ Slides exportados em PDF (`docs/apresentacao/slides_preliminar.md` → Marp → PDF)
- ☐ Cronometrado ≤ 5 min; cada um sabe sua frase do slide 5
- ☐ Nenhuma senha/token na tela; terminal com fonte grande
- ☐ Nomes da equipe preenchidos no README e no slide 1